

While you're working if you run into any conflicts or challenges, write them here. Explain what you did to over-came, or if you gave up.

Include any references & citations here, and within your code.

Aaron M

September 22, 2025

Work done:

- Initialized the Godot 4.5.1 project environment and established the repository structure under `NABG_godot`.
- Configured GitHub Desktop for version control, creating the initial repository and commit.
- Implemented the basic `proto_controller` scene and script to serve as the first playable prototype.
 - Set up a `CharacterBody3D` root node with camera control, collision shape, and simple WASD movement.
 - Added initial jumping and gravity logic to test player mobility.
- Verified the engine's default input map and ensured player controls responded correctly in the test scene.
- Shared the functional base with the team to enable further enhancement and iteration.

Challenges:

- Encountered inconsistencies in input responsiveness due to missing key bindings during early testing.

Resolution:

- Added basic input actions manually under Project Settings → Input Map to ensure movement keys were properly recognized.

References:

- Godot Docs 4.5: [Setting up the game area](#)
 - Godot Docs 4.5: [Listening to Player Input](#)
 - <https://godotshaders.com/shader/vhs-and-crt-monitor-effect/>
 - https://www.youtube.com/watch?v=ENhYj_VDDcg
 - <https://www.youtube.com/watch?v=mx6ycRSgk58>
 - <https://www.youtube.com/watch?v=xAO3x-Uhfoo>
 - <https://www.mediafire.com/file/7pxiguah1d927o0/horrorambiance3.mp3/file>
 -
-

Justin N

Date October 24, 2025

<https://www.youtube.com/watch?v=weyNRSLGfjE>

Work done:

- Constructed the Main Menu Scene as the first thing players would see
- Designed the main menu in a simple but yet appropriate manner that would convey the atmosphere of our game.
- Constructed the options scene: Only Master Volume functionality implemented
- Start button functionality implemented: Boots up the game
- Exit button functionality implemented: Exits the game
- Options button functionality implemented: directs the player to the options screen
- Added simple background music to main menu

Challenges:

- NA

Resolution:

- NA

References:

- Youtube: [Create MAIN MENU for your Godot game](#) - used as a reference to how to make a basic main menu.
- Youtube: [Creating volume sliders in Godot 4](#) - used as a reference on how to add volume sliders.

Date November 9th & 17, 2025

Work done:

- Constructed the Tutorial Scene prior to starting the game
- Constructed the Keybinds remapping inside of the settings and having those changes reflected and saved when exiting the game and coming back to it and be loaded.

Challenges:

- NA

Resolution:

- NA

References:

- Youtube: [Godot 4 Input Remapper Tutorial](#) - used as a reference to how to properly remap keybinds and save them.
- ChatGPT - assisted in properly recognizing the CTRL, ALT and SHIFT keys as a proper input to be remapped to.

Justin S

October 19-20, 2025

Work done:

- Enhanced the existing **proto_controller** scene and script
- Refined the player movement system, improving responsiveness for acceleration, deceleration, sprinting, jumping, and freely.
- Adjusted the mouse look implementation to ensure consistent yaw and pitch behavior without jitter or drift.
- Restructured the node hierarchy of the player scene (**CharacterBody3D** → **Head** → **Camera3D**) for proper rotation handling and future scalability.
- Created initial **greyboxing** for sample rooms to test player scale, navigation flow, and collision boundaries before detailed modeling.
- Standardized project directory structure and naming conventions (snake_case, singular folders) across scenes, scripts, and resources to align with Godot and industry best practices.
- Implemented a new **GameState autoload singleton** (**autoload/game_state/gd**) to handle player death and respawn logic triggered by pitfalls.
- Created **Kill Volume** areas to detect player falls and reload the scene upon death.
- Validated **.gitignore** and **.gitattributes** functionality to ensure smooth version control and asset management.

Challenges:

- Encountered a “**GameState is not declared in current scope**” error due to the singleton not being registered in the project.
- Faced issues requiring **manual input configuration** for movement keys, which caused inconsistent control mapping when testing across environments.

Resolution:

- Resolved the issue by adding `res://autoload/game_state.gd` under **Project Settings** → **AutoLoad** with the name **GameState**, confirming global accessibility and successful function calls from kill volume scripts.
- Implemented a fallback **InputMap bootstrap** within the player script to automatically define default key bindings for core actions (`move_left`, `move_right`, `move_forward`, `move_back`, `jump`, `sprint`, and `freefly`), eliminating the need for manual setup in new project clones.

References:

- Godot Docs: [Autoload Singleton Setup](#)
- GitHub Copilot: Assisted in generating default input action bootstrap logic.
- YouTube (Brackeys): [How to Make 3D Games in Godot 4](#) – used as a reference for refining player movement behavior and greyboxing layout testing.

October 26, 2025

Work Done:

- Integrated and enhanced UI systems from team branches (**main menu**, **pause menu**, and **HUD**) for consistency before merging to main.
- Unified all pause and input logic under **GameState** to manage pausing, cursor visibility, and scene transitions.
- Improved **HUD layout** and **functionality** (health + stamina bars) with player signal connections and proper sizing.
- Cleaned redundant or conflicting code from earlier versions (old cursor capture, duplicate pause logic, unused autoloads).

Challenges:

- Fixed technical issues during merge, including node-type mismatches and inconsistent input handling across branches.
- Ensured UI elements (HUD, pause, menu) worked together seamlessly within the main gameplay scene.

Resolution:

- Verified stable integration of all UI components under unified project structure.

- Confirmed consistent pause/resume flow, functional HUD display, and smooth transition from main menu to gameplay.

References:

- Godot Docs : [Pausing games and process mode](#)
- Godot Docs: [CanvasLayer](#)
- GitHub Copilot: assisted with refining input handling and scene-change logic.

October 27-28, 2025

Work Done:

- Enhanced Anthony's HUD functionality (health and stamina) and pause system (Resume, Restart, Main Menu, and Quit options toggled with ESC).
- Implemented centralized GameState.gd handling pause, scene reload, spawn logic, and cursor visibility.
- Integrated SimpleDungeons addon (majikayogames/SimpleDungeons) for 3D procedural dungeon generation.
- Adjusted room and dungeon scale for accurate 1.8 m player height within 10 m voxel grid.
- Removed redundant signals, cleaned cursor and SceneTreeTimer code, inputmap revised, and restructured folders for clarity.
- Verified complete loop: **Main Menu** → **Dungeon Generation** → **Player Spawn** → **Gameplay Control**.

Challenges:

- Pause and cursor inconsistencies during scene transitions.
- Player needing to jump to climb stairs (floor snap misconfiguration).
- Input actions misconfigured on fresh project clones.
- Spawn timing issues after procedural generation completion.

Resolutions:

- Centralized pause logic in `GameState` with `paused_changed` signals and one-shot cursor capture.
- Adjusted `CharacterBody3D` floor settings for auto step-up movement.
- Implemented runtime InputMap setup to ensure universal key availability.
- Linked `dungeon_done_generating signal` → `GameState` to spawn player after generation completes.

References:

- GitHub: [majikayogames/SimpleDungeons](https://github.com/majikayogames/SimpleDungeons)
- GitHub Copilot: assisted with input and spawn logic refinements.

November 6-7, 2025

Work Done:

- Refactored the player controller into a cleaner, modular structure with consistent rotation handling (body yaw / head pitch) and predictable movement states.
- Implemented a stamina + exhaustion system that drains fully to zero, disables sprint reliably, and reactivates it only after passing a recovery threshold; in order to provide stable transitions between walking, sprinting, and exhaustion.
- Built a functional interaction pipeline using a RayCast3D and a modular **Interactable** base class, including a button component that triggers exit-teleportation through **GameState**.
- Updated **GameState** to maintain a persistent **active_spawn_point**, ensuring respawns and restarts always use the latest checkpoint rather than defaulting to the initial spawn.
- Added and applied new room textures from the **Backrooms PBR Texture Pack** (OpenGameArt.org), improving environmental clarity and aligning the level visuals with the intended liminal aesthetic.

Challenges:

- Stamina hovered above zero, causing sprint flicker and inconsistent movement.
- RayCast hit child meshes instead of the intended interactable, failing to call interaction logic.
- Procedural room instances caused exported exit-spawn references to be null.
- Spawn logic repeatedly reverted to the original start point due to outdated game-state updates.
- Attempted to use Blender software for texture and modeling, but the learning curve proved to be super steep.

Resolutions:

- Reordered stamina and movement conditions to produce reliable drain, recovery, and state switching.
- Added parent-traversal logic so RayCast results always resolve to the correct interactable node.
- Implemented a spawn-finder fallback in **GameState** to support procedurally generated rooms.

- Corrected how and when `active_spawn_point` updates so respawns consistently place the player at the latest checkpoint.
- Gave up using Blender and decided to use packaged textures from free online source and integrate them into Godot.

References:

- Godot Docs: CharacterBody3D, Signals, Autoloads, RayCast3D
- GitHub Copilot (2025): Assisted with controller and interaction-flow structure
- Backrooms PBR Texture Pack — [OpenGameArt.org](https://opengameart.org/content/backrooms-pbr-texture-pack)
(<https://opengameart.org/content/backrooms-pbr-texture-pack>)

November 25–26, 2025

Work Done:

- Built a modular `ceiling_light.tscn` with emissive panel and tuned `SpotLight3D` setup for soft fluorescent lighting.
- Corrected ceiling illumination:
 - Adjusted spotlight height, range, and cone angle
 - Balanced ambient lighting for tile visibility
 - Removed hotspot artifacts on the light panel
- Integrated background music for main menu and Level One using per-scene `AudioStreamPlayers`.
- Fully implemented Options audio settings:
 - Volume slider with live preview
 - Persistent session volume via `AudioSettings` autoload
 - Confirm/Back logic: Confirm saves changes, Back discards
 - Confirm button greys out until changes are made
- Fixed node path and signal connection issues in the Options scene.

Challenges:

- `SpotLights` initially failed to light ceiling tiles due to incorrect height and insufficient cone spread.
- Ambient settings oscillated between too dark and too bright, requiring multiple passes.
- Audio bus configuration in Godot 4.5 was non-obvious due to updated interface location.

Resolutions:

- Moved `SpotLights` downward and widened cone angle to distribute light evenly.
- Set moderate ambient energy for visibility without flattening shading.
- Configured a dedicated Music bus, `AudioSettings` autoload, and the new `MusicPlayer` autoload for clean track transitions and consistent volume control.

- Implemented robust confirm/discard logic for Options UI.
- Structured node references properly for stable initialization.

References:

- Godot Engine Documentation 4.5 (Lighting, Environment, Audio Buses, AudioServer API)
- Free Music Archive licensing guidelines (CC BY-SA 4.0)
- OpenAI ChatGPT (2025). Technical guidance and implementation support for lighting, audio systems, UI logic, and drafting the dev log entry.

November 27-28, 2025

Work Done:

- Added **GameMode.COMPLETE** and refactored `level_complete()` to centralize level-completion logic, pause handling, and music triggers.
- Updated **GameMenu** to show a dedicated **LevelCompleteRoot** through `_on_mode_changed()`, replacing the old timer workflow.
- Added a **Continue** button to ensure reliable progression while paused.
- Simplified **ExitDoor** so it only calls `GameState.level_complete()`.
- Reworked **interactable key setup** using `Area3D + CollisionShape3D` and baked CSG geometry into a clean `MeshInstance3D`.
- Integrated **six-footstep .OGG samples** and implemented walking/sprinting tempo, pitch variation, and `PROCESS_MODE_ALWAYS` for reliable playback.

Challenges:

- Paused-state input caused inconsistent confirmation on level completion.
- CSG-based models were not interactable without a proper physics root.
- Footstep audio was initially too quiet and required mix adjustments.

Resolutions:

- Adopted a **button-driven confirmation model** to ensure consistent level-complete interactions.
- Standardized interactables using `Area3D` roots and explicit collision.
- Boosted footstep volume through per-instance `volume_db` and improved SFX setup.

References:

- Godot 4.5 Documentation – SceneTree pausing, CollisionObject3D, CSG tools
- Project source files – `game_state.gd`, `game_menu.gd`, `exit_door.gd`, `AudioManager`, `proto_controller.gd`

- **ChatGPT assistance** – Refactoring level-complete workflow, interactable setup, audio integration, and drafting the dev log entry.

December 7, 2025

Work Done:

- Enhanced **Aaron's enemy prototype** by implementing a fully functional **chaser AI** with idle, chase, and kill behavior.
- Added **custom SFX** for idle, chasing, and enemy capture, with clean state-based transitions.
- Integrated the chaser into the core **GameState.kill_player("enemy_capture")** pipeline for consistent death handling.
- Implemented enemy spawning when the player picks up the key: player teleports to the exit, checkpoint updates, and chaser appears at the designated spawn point.
- Ensured the chaser resets correctly after death and continues pursuing the player on restart.
- Created a gameplay video for demonstration.

Challenges:

- Player movement locked after respawn due to leftover flags from the kill sequence.
- Enemy remained stuck in the KILLING state after first death.
- Chaser scene reference was missing on the key instance, preventing spawning.

Resolutions:

- Simplified kill flow to immediate `_kill_now()` using GameState logic only.
- Removed persistent KILLING state and ensured the AI returns to IDLE/CHASING after respawn.
- Assigned `chaser_scene` properly and verified spawn point detection through group lookup.

References:

- Aaron's initial enemy logic foundation.
- KillVolume and GameState death-handling pattern.
- Godot Docs 4.5 (Area3D, CharacterBody3D, autoload workflow).
- **ChatGPT** – assistance with debugging, state logic fixes, and dev log drafting.

December 8, 2025

Work Done:

- Reorganized the entire project into a clear, feature-based hierarchy.
- Refactored all autoloads (`GameState`, `AudioManager`, `AudioSettings`, `GameActions`) for consistency and reliability.
- Cleaned and standardized all gameplay scripts: player controller, interactables, key, exit door, enemy AI, kill volumes, and spawn hook.
- Updated UI scripts for menus, HUD, and options with consistent structure and callback handling.
- Rebuilt README.md to match the new structure and current features.

Challenges:

- Ensuring all scripts used the same interaction patterns and function signatures.
- Preventing system conflicts during the refactor (UI vs GameState vs Audio).
- Maintaining functional gameplay while reorganizing major script systems.

Resolutions:

- Unified all interactables to use a zero-argument `interact()` method.
- Standardized sectioning, naming, and error handling across every script.
- Strengthened GameState and Audio integration to remove inconsistencies

References:

- High Concept Document
- Dev Log
- Project codebase
- ChatGPT (OpenAI) – assistance with refactoring and documentation drafting.

Date

Work Done:

-

Challenges:

-

Resolutions:

-

References:

-
-

Anthony T

Date

Work done:

- Extending camera movement (up/down)
- Added sprinting
- Added stamina bar/sprinting
- Added healthbar
- Added pause menu

Challenges:

- Figuring out how to commit/push from another branch using github desktop

Resolution:

-

References:

- [#https://www.youtube.com/watch?v=JEQR4ALwVU](https://www.youtube.com/watch?v=JEQR4ALwVU) (for pause menu implementation)